



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**Analizador de Miele por
Espectrometría (HIVES)**



Presentado por Diego Alonso Soria
en Universidad de Burgos — 8 de junio
de 2026

Tutores: Carlos Cambra Baseca — Rubén Ruiz
González



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



D. Carlos Cambra Baseca, profesor del departamento de Digitalización. Y,
D. Rubén Ruiz González, profesor del departamento de Digitalización.

Expone:

Que el alumno D. Diego Alonso Soria, con DNI 71308187E, ha realizado el Trabajo final de Grado en Ingeniería Informática titulado Analizador de Mieles por Espectrometría (HIVES).

Y que dicho trabajo ha sido realizado por el alumno bajo la dirección de los que suscribe, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, 9 de junio de 2026

Vº. Bº. del Tutor:

Vº. Bº. del co-tutor:

D. Carlos Cambra Baseca

D. Rubén Ruiz González

Resumen

El presente Trabajo de Fin de Grado describe el desarrollo de **HIVES** (*Honey Identification and Verification by Spectroscopy*), un sistema ciberfísico de bajo coste para la caracterización botánica de la miel mediante espectrometría óptica e inteligencia artificial. La solución integra un sensor multicanal (410 nm–940 nm) con una aplicación de escritorio multiplataforma desarrollada en Python, que gestiona de forma concurrente la captura por puerto serie, la persistencia en SQLite, la inferencia predictiva y la generación de informes en PDF.

Durante el trabajo de campo se confeccionó un conjunto de datos empírico propio de **42 muestras** reales correspondientes a cinco variedades (Milflores, Retama, Miel de Bosque, Miel con Jaramago y Miel Industrial). Ante la escasez de muestras, se diseñó un flujo algorítmico que combina aumento de datos por difusión gaussiana y un estudio comparativo entre modelos de ensamblado (XGBoost, *Random Forest*) y redes neuronales densas (TensorFlow/Keras).

La aportación principal y verificada de este trabajo es la **integración funcional extremo a extremo** del hardware de captura, la capa de persistencia y el motor de inferencia, operando de forma estable y concurrente. Las métricas de clasificación obtenidas (XGBoost como modelo de referencia) deben interpretarse como una **línea base preliminar**: al haberse calculado mayoritariamente sobre datos aumentados sintéticamente a partir de 42 muestras reales, no constituyen aún evidencia de precisión agronómica, sino una validación de la viabilidad técnica del prototipo. El trabajo sienta así una base de ingeniería sólida y documenta de forma explícita las condiciones necesarias para una validación estadística futura sobre datos reales masivos.

Descriptores

Espectrometría, Inteligencia Artificial, Aprendizaje Automático, Miel, Python, Sistemas Ciberfísicos, PyQt6, TensorFlow, XGBoost.

Abstract

This Bachelor’s Thesis describes the development of **HIVES** (Honey Identification and Verification by Spectroscopy), a low-cost cyber-physical system for the botanical characterization of honey using optical spectrometry and artificial intelligence. The system integrates a multichannel sensor (410 nm–940 nm) with a cross-platform desktop application written in Python that concurrently handles serial data capture, SQLite persistence, predictive inference and PDF report generation.

A proprietary empirical dataset of **42 real samples** across five varieties was collected. Given the scarcity of samples, a pipeline combining Gaussian-noise data augmentation and a comparative study of ensemble models (XGBoost, Random Forest) and dense neural networks (TensorFlow/Keras) was designed.

The main **verified** contribution is the end-to-end functional integration of the capture hardware, the persistence layer and the inference engine, running stably and concurrently. The reported classification metrics (with XGBoost as the reference model) must be read as a **preliminary baseline**: as they were computed mostly on data synthetically augmented from 42 real samples, they do not yet constitute evidence of agronomic accuracy, but rather validate the technical feasibility of the prototype. The work thus establishes a solid engineering foundation and explicitly documents the conditions required for future statistical validation on large real datasets.

Keywords

Spectrometry, Artificial Intelligence, Machine Learning, Honey, Python, Cyber-Physical Systems, PyQt6, TensorFlow, XGBoost.

Índice general

Índice general	iii
Índice de figuras	v
Índice de tablas	vi
1. Introducción	1
1.1. Estructura de la memoria	2
1.2. Estructura de los apéndices y materiales entregados	3
2. Objetivos del proyecto	5
2.1. Objetivos del producto software	5
2.2. Objetivos técnicos	6
3. Conceptos teóricos	9
3.1. Espectrometría y caracterización de mieles	9
3.2. Inteligencia Artificial y modelado predictivo	11
3.3. Integración Hardware-Software	13
4. Técnicas y herramientas	15
4.1. Metodología de desarrollo	15
4.2. Tecnologías principales	16
4.3. Comunicación hardware-software	18
4.4. Herramientas de soporte	18
5. Aspectos relevantes del desarrollo del proyecto	21
5.1. Construcción empírica del conjunto de datos y protocolo de muestreo	21

5.2. Estudio comparativo, generación de datos sintéticos y estrategia algorítmica	23
5.3. Análisis crítico de las limitaciones estadísticas del modelo . .	27
6. Trabajos relacionados	31
6.1. Caracterización física y colorimetría espectral tradicional . .	31
6.2. Espectroscopía infrarroja y detección de adulteraciones . . .	32
6.3. Evolución hacia repositorios abiertos y modelos de Machine Learning	32
6.4. Posicionamiento y aportación del sistema HIVES	33
6.5. Técnica Híbrida de modelos de ML (XGBoost + TensorFlow)	34
7. Conclusiones y Líneas de trabajo futuras	35
7.1. Conclusiones	35
7.2. Líneas de trabajo futuras	36
Bibliografía	39

Índice de figuras

3.1. Regiones del espectro electromagnético incluyendo los 18 canales del sensor AS7265x (410 nm – 940 nm). Fuente: [6]	10
3.2. Módulo SparkFun Triad AS7265x empleado como núcleo espectrométrico del sistema HIVES. Fuente: [23]	10
3.3. Arquitectura de la red neuronal densa (MLP) implementada en las primeras pruebas de HIVES. Fuente: Elaboración Propia. . .	12
5.1. Cadena de comunicación hardware-software del sistema HIVES: desde el sensor AS7265x hasta la capa de inferencia de la aplicación de escritorio. Fuente: Elaboración Propia.	23

Índice de tablas

5.1. Comparativa de los modelos evaluados en HIVES. La exactitud se obtuvo mediante validación cruzada de 5 pliegues. Los tiempos de entrenamiento corresponden a la ejecución paralela asistida por GPU en Google Colab.	27
---	----

Introducción

La viabilidad comercial de las mieles monoflorales se ve amenazada por el fraude alimentario y la adulteración, motivados por su alto valor en el mercado en comparación con las variantes multiflorales. En la actualidad, el método analítico de referencia para verificar el origen botánico de la miel es el análisis melisopalinológico. Sin embargo, esta técnica tradicional presenta severas limitaciones operativas en los entornos productivos modernos: requiere personal altamente especializado en botánica, exige instrumental óptico avanzado y conlleva tiempos de procesamiento prolongados. Estas ineficiencias logísticas y económicas hacen imperativa la investigación de metodologías alternativas y automatizables que provean un cribado analítico ágil y de bajo coste.

Para dar respuesta a esta necesidad, el presente Trabajo de Fin de Grado propone el diseño y desarrollo integral del sistema **HIVES** (*Honey Identification and Verification by Spectroscopy*), una solución ciberfísica modular que fusiona la espectrometría óptica de bajo coste con técnicas de Inteligencia Artificial para automatizar la clasificación botánica de las mieles de forma no destructiva. La arquitectura hardware se fundamenta en un sensor espectrométrico multicanal (**AS7265x**) capaz de registrar firmas ópticas discretas en 18 bandas espectrales distribuidas entre las regiones del ultravioleta, el espectro visible y el infrarrojo cercano.

Los flujos de datos capturados se transmiten mediante un protocolo de comunicación serie **UART** hacia una aplicación de escritorio multiplataforma. Este software, desarrollado íntegramente en **Python 3.10** bajo el framework gráfico reactivo **PyQt6**, gestiona la persistencia histórica mediante el motor de base de datos local embebido **SQLite**. A nivel de rendimiento, el sistema implementa concurrencia asíncrona a través de hilos de ejecución secundarios para garantizar la fluidez de la interfaz de usuario. Finalmente, la plataforma

integra un entorno de inferencia predictiva dual que evalúa y clasifica las muestras basándose tanto en modelos de aprendizaje automático clásico (XGBoost) como en arquitecturas de aprendizaje profundo preentrenadas (TensorFlow y Keras).

Todo el código fuente desarrollado a lo largo de este trabajo se encuentra en un repositorio de GitHub compartido con el tribunal, al cual se puede acceder a través del siguiente enlace:

<https://github.com/Das-dev24/Trabajo-Fin-de-Grado-2025-2026>

El progreso del proyecto se ha registrado en la plataforma Jira, el panel creado para ello también se encuentra compartido con el tribunal, el enlace de acceso al mismo es el siguiente:

<https://das-team-hives.atlassian.net/jira/software/projects/SCRUM/boards/1>

1.1. Estructura de la memoria

El cuerpo principal de esta memoria técnica se organiza en siete capítulos estructurados lógicamente para argumentar las decisiones de ingeniería de software adoptadas a lo largo de la investigación:

- **Capítulo 1: Introducción:** Define el contexto socioeconómico, la motivación del proyecto, la problemática de los métodos tradicionales en el sector apícola y ofrece una descripción general de la solución HIVES desarrollada, delimitando la estructura documental.
- **Capítulo 2: Objetivos del proyecto:** Desglosa detalladamente el alcance del proyecto, clasificándolo de forma normalizada entre objetivos del producto software (funcionalidades y comportamiento esperado del sistema) y objetivos técnicos (retos formativos, metodológicos e instrumentales de implementación).
- **Capítulo 3: Conceptos teóricos:** Establece el marco científico del proyecto, abarcando los principios de la física óptica, la caracterización bromatológica melífera, la taxonomía del aprendizaje supervisado, las redes neuronales densas, el modelado clásico mediante bosques aleatorios y las bases de las comunicaciones serie.

- **Capítulo 4: Técnicas y herramientas:** Ofrece una justificación técnica exhaustiva y comparativa de la totalidad de la pila tecnológica seleccionada frente a las opciones descartadas, incluyendo metodologías ágiles (Scrum y Jira), lenguajes de programación, frameworks de interfaz gráfica y sistemas de persistencia.
- **Capítulo 5: Aspectos relevantes del desarrollo del proyecto:** Describe el núcleo de la experiencia práctica del desarrollo de la solución, detallando el protocolo físico para la confección empírica del *dataset* de 42 muestras y argumentando detalladamente el diseño del flujo algorítmico dual implementado para sortear la escasez de datos.
- **Capítulo 6: Trabajos relacionados:** Expone una revisión sistemática del estado del arte, analizando críticamente otras soluciones técnicas, patentes y proyectos académicos previos enfocados en la caracterización espectroscópica de alimentos y la detección de fraudes.
- **Capítulo 7: Conclusiones y líneas de trabajo futuras:** Sintetiza el balance técnico final y el grado de consecución del proyecto, analizando críticamente las lecciones aprendidas sobre concurrencia y comunicación hardware, y planteando la hoja de ruta para la evolución tecnológica del prototipo.

1.2. Estructura de los apéndices y materiales entregados

Complementariamente a la memoria principal, el proyecto se acompaña de un volumen independiente de anexos técnicos estructurado de forma estricta en seis apéndices obligatorios que recogen los artefactos propios de la disciplina de la Ingeniería del Software:

- **Apéndice A: Plan de Proyecto Software:** Documenta la gestión ágil del ciclo de vida del proyecto mediante la recopilación de los catálogos de tareas y el análisis de los gráficos *burn-down* correspondientes a las 10 iteraciones ejecutadas en Jira , concluyendo con el estudio de viabilidad técnica, temporal, económica y legal.
- **Apéndice B: Especificación de Requisitos:** Contiene el catálogo formalizado de los 11 requisitos funcionales y 9 requisitos no funcionales del sistema , detallando el comportamiento dinámico esperado a

través de cuadros estructurados de Casos de Uso como CU-01 (Analizar Muestra), CU-02 (Generar Reporte PDF) y CU-03 (Consultar Histórico).

- **Apéndice C: Especificación de diseño:** Detalla las decisiones arquitectónicas bajo el patrón conceptual MVC , el modelo entidad-relación y diccionario de tablas de la base de datos de persistencia local , acompañados de los diagramas procedimentales *UML* de secuencia y flujo de datos (*DFD*).
- **Apéndice D: Documentación técnica de programación:** Conforma el manual de ingeniería dirigido a desarrolladores, describiendo la jerarquía modular de directorios del repositorio , la configuración del entorno virtual de dependencias, las directrices de compilación independiente mediante *PyInstaller* y la batería de pruebas de integración física aplicadas.
- **Apéndice E: Documentación de usuario:** Provee el manual de operación final del sistema para usuarios sin conocimientos técnicos avanzados , detallando de forma visual los requisitos de instalación, la secuencia de despliegue y los flujos de interacción con la interfaz gráfica reactiva.
- **Apéndice F: Sostenibilización curricular:** Justifica la alineación e impacto del presente proyecto con los Objetivos de Desarrollo Sostenible de la Agenda 2030, junto con una reflexión ética y de responsabilidad profesional sobre las implicaciones del sistema desarrollado.

La memoria principal y los anexos forman un conjunto completo que documenta todo el ciclo de ingeniería del software, desde la teoría que justifica el diseño hasta las pruebas del prototipo. El objetivo es ofrecer al lector una visión detallada y clara del proceso, asegurando que cada decisión técnica en el desarrollo de HIVES esté bien fundamentada y respaldada. Siguiendo esta estructura, los próximos capítulos detallan los fundamentos científicos, las tecnologías utilizadas y los aspectos prácticos que hicieron posible esta solución ciberfísica.

Objetivos del proyecto

El presente Trabajo de Fin de Grado tiene su origen en una propuesta planteada por el equipo de tutorización académica de la Escuela Politécnica Superior, aceptada por el alumno motivado por su interés particular en profundizar en el estudio de las redes neuronales y los algoritmos de *Machine Learning*. La aplicación de estas técnicas a un caso de uso real, como es el análisis de muestras de miel mediante espectrometría, ha permitido conjugar un objetivo formativo de especialización técnica con la resolución de una problemática concreta del sector apícola, en el cual los análisis convencionales suelen presentar costes elevados y tiempos de procesamiento prolongados.

Conforme a la estructura habitual de proyectos de Ingeniería del Software, los objetivos planteados en este TFG se han clasificado en dos categorías diferenciadas: por un lado, los **objetivos del producto software**, que definen el comportamiento esperado del sistema resultante; y por otro, los **objetivos técnicos**, que recogen los retos formativos y de implementación abordados durante el desarrollo del proyecto.

2.1. Objetivos del producto software

Los objetivos del producto definen el alcance funcional del sistema HIVES (*Honey Identification and Verification by Spectroscopy*), cuya especificación detallada se encuentra en el Apéndice B. A continuación, se presenta una síntesis de los mismos:

OP-1: Automatización del proceso de análisis.

Habilitar la captura automatizada de los datos espectrales mediante la

integración del software con un dispositivo espectrométrico, eliminando la necesidad de intervención manual durante la lectura de las muestras.

OP-2: Clasificación inteligente mediante técnicas de IA.

Implementar un sistema de inferencia basado en una red neuronal preentrenada con **TensorFlow** o **Scikit-Learn**, capaz de discriminar el tipo de miel a partir de su firma espectral con un nivel de confianza cuantificable.

OP-3: Generación automatizada de informes técnicos.

Producir reportes en formato **PDF** que recojan la totalidad de la información relevante de cada análisis (fecha, resultados de clasificación, métricas espectrales y métricas de confianza del modelo), facilitando su archivo y distribución.

OP-4: Persistencia y trazabilidad del histórico.

Almacenar los datos de cada análisis en una base de datos local, garantizando su consulta posterior y posibilitando la extracción de conjuntos de datos para futuros reentrenamientos del modelo.

OP-5: Usabilidad y accesibilidad operativa.

Proporcionar una interfaz gráfica intuitiva que permita el manejo del sistema por parte de personal sin conocimientos técnicos avanzados, asegurando además la compatibilidad multiplataforma.

2.2. Objetivos técnicos

Los objetivos técnicos recogen los principales retos formativos, metodológicos y de implementación que se han afrontado durante el desarrollo del proyecto. Estos objetivos no se traducen directamente en funcionalidades del producto, sino que constituyen el ámbito de aprendizaje y especialización del alumno como ingeniero informático.

OT-1: Profundizar en el estudio de algoritmos de *Machine Learning*.

Adquirir un conocimiento sólido sobre los fundamentos teóricos y prácticos del aprendizaje automático, abordando la totalidad del ciclo de vida del modelo: preparación y normalización de los datos, diseño de la arquitectura de la red, entrenamiento, evaluación de métricas e integración del modelo entrenado en una aplicación de producción.

OT-2: Emplear *Jupyter Notebooks* como entorno de experimentación.

Utilizar el ecosistema de cuadernos interactivos para llevar a cabo las fases de exploración de los datos espectrales, prototipado de arquitecturas de red, entrenamiento iterativo y evaluación comparativa de los modelos, separando de forma clara la fase de investigación de la posterior fase de integración en la aplicación final.

OT-3: Integrar la comunicación hardware-software a bajo nivel.

Establecer una interconexión fiable entre la aplicación de escritorio y el dispositivo espectrométrico mediante el protocolo de comunicación por puerto serie, gestionando la sincronización, los tiempos de espera y los escenarios de desconexión abrupta del hardware.

OT-4: Aplicar el patrón arquitectónico Modelo-Vista-Controlador (MVC).

Diseñar e implementar el sistema siguiendo una arquitectura modular que garantice una separación estricta entre la interfaz gráfica, la lógica de negocio y la capa de datos, favoreciendo la mantenibilidad y la escalabilidad del software.

OT-5: Gestionar la concurrencia en aplicaciones de escritorio.

Implementar un modelo de ejecución multihilo que permita delegar las tareas de adquisición de datos y de inferencia del modelo a hilos secundarios, evitando el bloqueo de la interfaz gráfica (*GUI freezing*) durante las operaciones intensivas.

OT-6: Aplicar una metodología ágil real en un proyecto uni-personal.

Llevar a la práctica el marco de trabajo **Scrum** apoyado en la herramienta **Jira**, asumiendo la gestión completa del *Product Backlog*, la planificación de *sprints*, la estimación mediante *Story Points* y el análisis de las métricas de velocidad y *burn-down*.

OT-7: Garantizar la compatibilidad multiplataforma.

Diseñar la aplicación de forma que pueda ejecutarse de manera consistente sobre los sistemas operativos Windows, Linux y macOS, gestionando adecuadamente las particularidades de cada entorno en lo relativo al acceso al puerto serie y al sistema de archivos.

OT-8: Diseñar una capa de persistencia local eficiente.

Implementar el almacenamiento de los datos mediante el motor **SQLite**, prescindiendo del uso de mapeadores objeto-relacional (ORM) con el fin de optimizar el rendimiento y mantener un control directo sobre las sentencias SQL.

OT-9: Empaquetar y distribuir la aplicación como ejecutable independiente.

Generar un binario autónomo que encapsule el intérprete de Python, las dependencias y el modelo entrenado, permitiendo al usuario final ejecutar el sistema sin requerir instalaciones técnicas previas.

La consecución conjunta de los objetivos del producto y los objetivos técnicos descritos constituye la base sobre la que se ha articulado el diseño, la implementación y la validación del sistema HIVES, cuyos detalles se desarrollan en los apéndices posteriores.

Conceptos teóricos

El sistema HIVES integra conocimientos de espectroscopía óptica, bromatología apícola, inteligencia artificial e ingeniería de sistemas embebidos. Este capítulo sintetiza los fundamentos técnicos que justifican las decisiones de diseño adoptadas durante el desarrollo del proyecto.

3.1. Espectrometría y caracterización de mieles

La viabilidad comercial de las mieles monoflorales, determinada por su origen botánico, exige métodos de verificación que prevengan el fraude alimentario [17]. Aunque el análisis melisopalínológico [16] constituye el estándar normativo, sus altos costes y tiempos de procesamiento invitan a la búsqueda de una alternativa automatizada.

Para dar respuesta a esta necesidad, el sistema emplea la espectrometría de reflectancia difusa, que cuantifica la interacción de la luz con la materia para obtener una firma espectral única [22]. El hardware seleccionado es el sensor **SparkFun Triad AS7265x**, capaz de capturar la intensidad reflectiva en 18 canales discretos (entre 410 nm y 940 nm), abarcando los espectros ultravioleta (UV), visible y el infrarrojo cercano (NIR) [1].

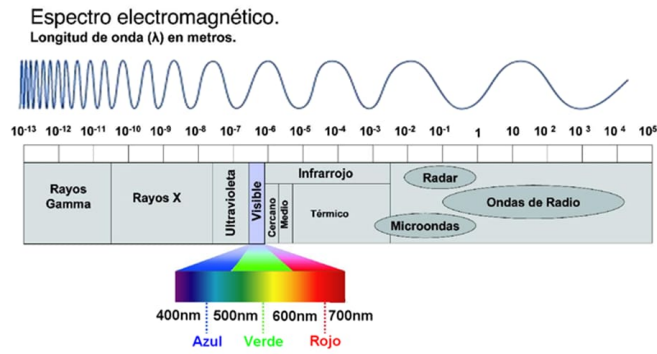


Figura 3.1: Regiones del espectro electromagnético incluyendo los 18 canales del sensor AS7265x (410 nm – 940 nm). Fuente: [6]

Esta representación espectral compacta permite entrenar modelos predictivos para identificar las clases botánicas evaluadas en este proyecto: **Milflores**, **Retama**, **Miel de Bosque**, **Miel con Jaramago** y **Miel Industrial**.

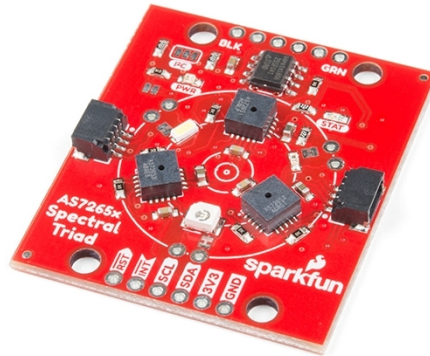


Figura 3.2: Módulo SparkFun Triad AS7265x empleado como núcleo espectral del sistema HIVES. Fuente: [23]

3.2. Inteligencia Artificial y modelado predictivo

El núcleo analítico de HIVES se fundamenta en algoritmos de aprendizaje supervisado, diseñados para inferir la clase botánica a partir de la firma espectral normalizada (mediante escalado Min-Max o Z-score, garantizando una convergencia estable).

Redes Neuronales Densas (MLP)

La arquitectura principal consiste en un Perceptrón Multicapa (MLP) adaptado a la dimensionalidad del sensor multicanal. La salida de cada neurona se computa aplicando una función de activación f a la suma ponderada de sus entradas y su sesgo:

$$y = f \left(\sum_{i=1}^n w_i x_i + b \right) \quad (3.1)$$

Donde:

- y : salida de la neurona.
- f : función de activación, que introduce la no linealidad.
- x_i : valor de la i -ésima entrada.
- w_i : peso asociado a la entrada x_i .
- b : sesgo (*bias*), que desplaza la suma ponderada.
- n : número total de entradas de la neurona.

La red cuenta con una capa de entrada de 18 neuronas y tres capas ocultas (512, 256 y 128 neuronas). En estas últimas se emplea la función de activación **Swish** [19], seleccionada por su robusta propagación del gradiente en arquitecturas de mediana profundidad.

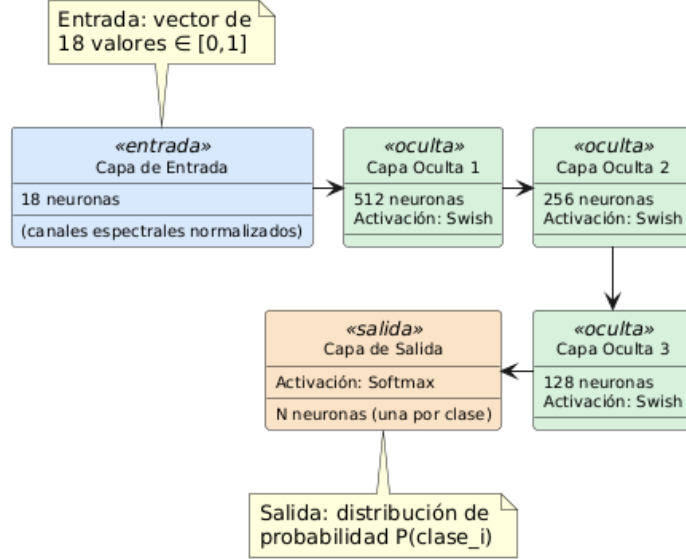


Figura 3.3: Arquitectura de la red neuronal densa (MLP) implementada en las primeras pruebas de HIVES. Fuente: Elaboración Propia.

Para la clasificación multiclase, la capa de salida utiliza la función **Softmax**, que transforma las predicciones en una distribución de probabilidad sobre las K clases posibles:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \quad (3.2)$$

El entrenamiento minimiza la entropía cruzada categórica ($\mathcal{L}$) mediante el optimizador **Adam** [15], actualizando iterativamente los pesos en base al gradiente:

$$\mathcal{L} = - \sum_{i=1}^K y_i \log(\hat{y}_i) \quad (3.3)$$

$$w_i \leftarrow w_i - \eta \cdot \frac{\partial \mathcal{L}}{\partial w_i} \quad (3.4)$$

Bosques Aleatorios y prevención del sobreajuste

La escasez de datos empíricos etiquetados en el dominio apícola incrementa el riesgo de sobreajuste (*overfitting*). En la red neuronal, este fenómeno se mitiga mediante técnicas de regularización, *Dropout* y *Early Stopping*.

Como estrategia complementaria, se implementa un modelo de **Bosques Aleatorios** (*Random Forest*) [4]. Este método de ensamblado construye múltiples árboles de decisión y combina sus predicciones. A través de la inyección de aleatoriedad (*bagging* y selección aleatoria de características), el algoritmo reduce drásticamente la varianza, ofreciendo un rendimiento robusto en regímenes con pocas muestras. La evaluación integral de todos los modelos se realiza mediante validación cruzada por pliegues (*k-fold cross-validation*), asegurando que las métricas de rendimiento sean estadísticamente significativas [11].

3.3. Integración Hardware-Software

La adquisición de espectros requiere una comunicación determinista entre la aplicación de escritorio y el hardware de medición. Este enlace físico se establece mediante el protocolo asíncrono **UART**, canalizado a través del bus USB mediante un microcontrolador que actúa como pasarela. Esta arquitectura simplifica notablemente el despliegue del sistema operativo, ya que los entornos modernos (Windows, Linux, macOS) instancian el puerto serie virtual de forma nativa, evitando la instalación de *drivers* propietarios en el equipo del usuario final [2].

Técnicas y herramientas

El desarrollo de HIVES ha requerido la evaluación y selección de un ecosistema heterogéneo de tecnologías. Este capítulo documenta y justifica las decisiones adoptadas en cada ámbito funcional, priorizando la mantenibilidad, el rendimiento, la compatibilidad multiplataforma y la viabilidad de despliegue.

4.1. Metodología de desarrollo

La planificación del proyecto se ha articulado en torno a **Scrum**, marco de trabajo ágil iterativo e incremental consolidado como estándar *de facto* en la industria. Dado el carácter unipersonal del TFG, se ha aplicado una adaptación metodológica en la que el alumno asume simultáneamente los roles de *Product Owner*, *Scrum Master* y equipo de desarrollo, mientras que los tutores académicos ejercen como *stakeholders* proporcionando validación y retroalimentación. El ciclo de desarrollo se ha discretizado en *sprints* de siete días naturales, ventana que equilibra la necesidad de presentar incrementos evaluables con la agilidad necesaria para pivotar ante imprevistos técnicos.

Como infraestructura de soporte metodológico se ha empleado **Jira**. Todo el alcance funcional se ha modelado en un *Product Backlog* estructurado en Épicas y Tareas, con estimación mediante Puntos de Historia (*Story Points*). Los gráficos de trabajo pendiente (*burn-down charts*) generados automáticamente han permitido auditar la velocidad de desarrollo y la adherencia a la planificación en cada iteración.

4.2. Tecnologías principales

Lenguaje de programación: Python

La totalidad del ecosistema software de HIVES se ha programado en **Python 3.10**. La decisión responde al paradigma de **unificación tecnológica**: en sistemas que integran flujos de aprendizaje automático con una capa de presentación visual, disociar el lenguaje del modelo del lenguaje de la aplicación introduce sobrecoste de serialización, latencias de comunicación y complejidades en el despliegue. Al ser Python el lenguaje hegemónico en la comunidad científica de Inteligencia Artificial, su extensión a la capa de presentación y a la lógica de negocio elimina estas barreras. Adicionalmente, su biblioteca estándar cubre las necesidades del proyecto: concurrencia mediante hilos, comunicación serie asíncrona y manipulación matricial de datos espectrales.

Motor de inferencia: destilación de conocimiento con TensorFlow y XGBoost

El diseño del motor de clasificación ha atravesado un proceso evolutivo que culmina en una arquitectura híbrida, estrategia que cuenta con respaldo en la literatura reciente: [14] demuestra que la combinación de enfoques complementarios dentro de un marco híbrido mejora la capacidad predictiva frente al empleo de un único modelo aislado. Durante las fases de evaluación algorítmica se constató empíricamente que los modelos basados en ensamblados de árboles de decisión —en particular **XGBoost** [5]— exhibían una capacidad de generalización superior a la de las redes neuronales puras cuando el espacio muestral es reducido, circunstancia inherente al dominio del proyecto. Sin embargo, el despliegue directo de un modelo XGBoost introducía fricciones arquitectónicas en el módulo de inferencia, concebido para operar con grafos computacionales de **TensorFlow/Keras** [9].

Para ello se implementó una estrategia de **destilación de conocimiento** (*knowledge distillation*) [12], que discurre en dos etapas:

1. **Transferencia de conocimiento.** Un clasificador maestro XGBoost, calibrado mediante búsqueda exhaustiva de hiperparámetros (*Grid Search*), procesa el conjunto de entrenamiento para generar vectores de probabilidad suavizada (*soft labels*). Estas distribuciones retienen información topológica sobre la similitud espectral entre clases, aportando

un gradiente de aprendizaje más rico que las etiquetas categóricas binarias.

2. **Imitación tensorial.** Una red neuronal densa (estudiante) programada en Keras se entrena para mimetizar las predicciones del modelo maestro. La topología emplea capas de compresión dimensional ($128 \rightarrow 64 \rightarrow 32$) con activaciones no lineales y una capa de salida *Softmax*, optimizando la entropía cruzada categórica frente a las *soft labels*.

El resultado es un modelo neuronal ligero y determinista que incorpora la heurística extraída del ensamblado, distribuible en un único fichero `.keras` sin dependencias del entorno de entrenamiento.

Interfaz gráfica y concurrencia: PyQt6

La capa de presentación se ha materializado con **PyQt6**, los *bindings* oficiales del *framework* C++ Qt 6 para Python. Frente a alternativas como Tkinter o wxPython, PyQt6 garantiza aspecto nativo en los tres sistemas operativos objetivo (Windows, Linux y macOS) y proporciona dos pilares críticos para la arquitectura:

- **Patrón Modelo-Vista-Controlador**, facilitado por el mecanismo de **señales y slots** (*signals & slots*), que desacopla completamente el estado interno de la aplicación de su representación visual.
- **Concurrencia sin bloqueo**, mediante la clase **QThread**, que delega los procesos de entrada/salida bloqueantes —lectura del puerto serie y cálculo de la predicción— a hilos en segundo plano, preservando la fluidez del bucle de eventos principal y eliminando la congelación de la interfaz.

Persistencia local: SQLite

El histórico de análisis se gestiona con el motor relacional **SQLite**. Su adopción obedece a tres directrices de diseño: inmediatez, portabilidad y aislamiento. Al carecer de arquitectura cliente-servidor, SQLite embebe el motor transaccional en la propia aplicación, materializando la base de datos en un único fichero local sin dependencias de red ni gestión de credenciales. Para minimizar la sobrecarga computacional en las rutinas de inserción y recuperación, se ha descartado el uso de mapeadores objeto-relacional (ORM), accediendo a los datos mediante sentencias SQL directas.

4.3. Comunicación hardware-software

La integración del sensor espectrométrico con la aplicación se segmenta en dos niveles. En la capa de adquisición física, un microcontrolador Arduino Uno actúa como pasarela de conversión de protocolos, interrogando al sensor AS7265x a través del bus **I²C** y empaquetando los 18 canales espectrales para su transmisión. En el lado del equipo informático, la biblioteca **PySerial** instancia un puente de comunicación asíncrona con el puerto serie virtual enumerado sobre USB, gestionando los búferes de entrada para evitar pérdida de tramas durante la transferencia de datos.

4.4. Herramientas de soporte

El resto de herramientas que completan el ecosistema del proyecto se reseñan a continuación:

- **Google Colab (Jupyter Notebooks).** Las fases de exploración analítica, *data wrangling* y calibración de modelos se han desarrollado en Jupyter Notebooks sobre Google Colab, cuya disponibilidad de GPU ha acelerado exponencialmente los ciclos de convergencia en comparación con el cómputo local en CPU.
- **FPDF2.** La generación automatizada de informes periciales en PDF se delega en esta biblioteca, que permite la composición directa de primitivas gráficas, tablas dinámicas y texto estructurado con una huella de memoria mínima, evitando dependencias de renderizadores HTML/CSS.
- **Git y GitHub.** El control de versiones se ha gestionado con Git, con el repositorio alojado en GitHub. Esta combinación garantiza la trazabilidad de cada incremento del proyecto y facilita la colaboración con los tutores.
- **SonarQube.** El análisis estático continuado del código fuente mediante SonarQube ha permitido auditar la deuda técnica, las vulnerabilidades de seguridad y los *code smells*, asegurando el cumplimiento de los estándares *Clean Code*.
- **PlantUML.** El modelado arquitectónico en UML se ha realizado como código declarativo, garantizando la compatibilidad de los diagramas con el sistema de control de versiones e impidiendo la degradación visual de la documentación.

- **PyInstaller.** El empaquetado final de la aplicación —intérprete Python, modelo tensorial, base de datos y dependencias— en un único ejecutable *standalone* se realiza con PyInstaller, eliminando cualquier proceso de instalación de dependencias para el usuario final.
- **LaTeX y Overleaf.** La redacción de la presente memoria y sus anexos se ha llevado a cabo en L^AT_EX operado a través de Overleaf, plataforma que facilita las rondas iterativas de revisión académica.

Aspectos relevantes del desarrollo del proyecto

En este capítulo se exponen y justifican las decisiones técnicas, metodológicas e instrumentales más críticas adoptadas durante la ejecución práctica del sistema HIVES. En este apartado se argumentarán detalladamente las soluciones de ingeniería de software aplicadas para resolver las problemáticas reales detectadas durante el transcurso del proyecto, prestando especial atención a la interacción física con el sensor óptico y a las severas restricciones algorítmicas impuestas por el volumen del conjunto de datos empírico disponible.

5.1. Construcción empírica del conjunto de datos y protocolo de muestreo

Uno de los desafíos prácticos más significativos y complejos del presente Trabajo de Fin de Grado ha sido la creación, limpieza y consolidación de un conjunto de datos (*dataset*) propio y representativo. A diferencia de múltiples proyectos de aprendizaje automático convencional que se nutren de repositorios masivos preexistentes o de simulaciones públicas estandarizadas, el correcto funcionamiento del sistema HIVES requería que el conjunto de datos de entrenamiento fuese obtenido nativamente por el propio dispositivo diseñado.

Para garantizar la reproducibilidad analítica y asegurar que el sensor multicanal AS7265x capturara firmas espectrales estables y consistentes, se diseñó un estricto protocolo de preparación fisicoquímica e instrumental de las muestras:

- **Acondicionamiento térmico y físico de la matriz:** Antes de realizar cualquier lectura espectral, cada muestra biológica de miel fue sometida a un proceso de tratamiento térmico sostenido a una temperatura controlada de 50°C durante un periodo ininterrumpido de 24 horas. La miel, al ser una solución acuosa sobresaturada de azúcares, presenta una alta tendencia termodinámica a la cristalización. Estos microcristales actúan como centros de dispersión óptica (fenómenos de dispersión de Mie y Rayleigh) que alteran drásticamente la reflectancia espectral. El tratamiento térmico resulta crítico para homogeneizar completamente la matriz fluida, estandarizar su índice de refracción y reducir las microburbujas de aire atrapadas, evitando así refracciones anómalas durante la incidencia de los haces de luz.
- **Calibración de referencia y supresión de corriente oscura (*Dark Current*):** Previo a la inserción de las muestras orgánicas, el sistema de captura fue sometido a rutinas de calibración de línea base. Esto incluyó la medición de la reflectancia y transmitancia en vacío (*blank calibration*) para aislar la interferencia de la luz ambiental residual, así como la cuantificación del ruido intrínseco de los fotodiodos en ausencia total de iluminación (*dark current correction*). Estos valores de compensación se sustrajeron algorítmicamente de las mediciones finales para garantizar la linealidad de la respuesta sensorica.
- **Disposición instrumental y aislamiento:** Tras finalizar la fase de acondicionamiento e igualar gradualmente la temperatura con el ambiente del laboratorio para evitar corrientes de aire, las muestras se transfirieron a cubetas estandarizadas de análisis óptico. Estas cubetas se posicionaron en el prototipo, diseñado específicamente para aislar la muestra de la contaminación lumínica externa.
- **Configuración y parametrización de la captura digital:** Se estableció un tiempo fijo de exposición activa e integración de 15 segundos por ciclo de lectura analógico-digital. Este intervalo de sobremuestreo (*oversampling*) permite promediar temporalmente las lecturas, mitigando el impacto del ruido de disparo (*shot noise*) inherente a los semiconductores del sensor. Durante esta ventana, el prototipo registró la respuesta espectral sobre los 18 canales discretos distribuidos entre los 410 nm y los 940 nm, abarcando las interacciones por reflectancia y transmitancia lumínica.

Como resultado directo de este exhaustivo trabajo de campo, condicionado fuertemente por las limitaciones temporales estrictas del calendario

académico y por la escasez logística de muestras certificadas mediante análisis melisopalinológico oficial, se consolidó un conjunto empírico final compuesto por **42 muestras útiles**. Aunque este tamaño poblacional es reducido bajo los estándares tradicionales de la ciencia de datos masivos, constituye un punto de partida empírico de incalculable valor para la validación funcional del hardware y la arquitectura de la información del sistema. Dichas mediciones se orchestaron siguiendo el esquema de comunicaciones ilustrado a continuación:

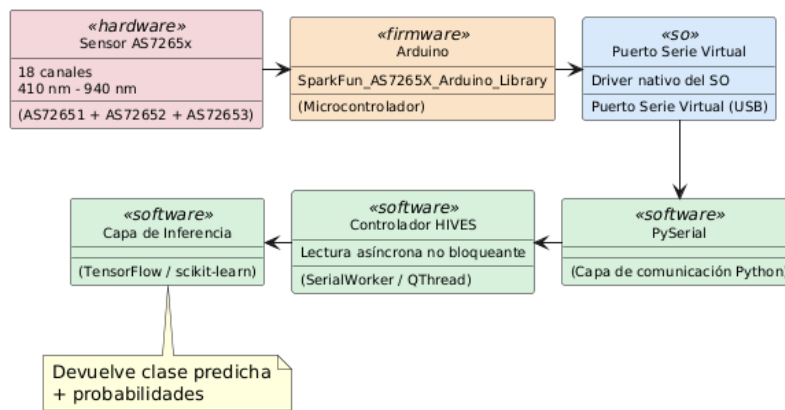


Figura 5.1: Cadena de comunicación hardware-software del sistema HIVES: desde el sensor AS7265x hasta la capa de inferencia de la aplicación de escritorio. Fuente: Elaboración Propia.

5.2. Estudio comparativo, generación de datos sintéticos y estrategia algorítmica

La disponibilidad de un régimen de escasez extrema de datos, limitado a únicamente 42 firmas espectrales, introduce un riesgo metodológico de primer orden en el modelado predictivo: el sobreajuste (*overfitting*) provocado por la *maldición de la dimensionalidad* (*curse of dimensionality*). Al proyectar 42 muestras sobre un espacio vectorial de 18 dimensiones (canales espectrales), el espacio queda escasamente poblado. Si estos vectores se inyectan directamente en un modelo matemático de caja negra de alta complejidad computacional, el sistema memorizará el ruido y las trazas específicas de cada muestra exacta, destruyendo por completo su capacidad de generalización operativa ante nuevas lecturas en un entorno real.

Para solventar esta problemática técnica aplicando principios de ingeniería de software robusta, se diseñó un flujo algorítmico estructurado en dos grandes fases complementarias: la síntesis controlada de datos y la evaluación comparativa de arquitecturas de aprendizaje profundo frente a modelos de ensamblado clásico.

Generación de datos sintéticos mediante difusión gaussiana

Ante la imposibilidad de expandir el tamaño muestral real, se aplicó una estrategia de **aumento de datos sintéticos** basada en la inyección de **ruido gaussiano** [3]. Esta técnica se fundamenta en la premisa de que las desviaciones instrumentales de un sensor óptico (ruido electrónico, ligeras variaciones en la intensidad de emisión del LED o micrométricas alteraciones en el posicionamiento de la cubeta) siguen típicamente una distribución estadísticamente normal.

El procedimiento consistió en tomar cada una de las 42 firmas espectrales reales como el vector de medias (centroide) de una distribución normal multivariante. A partir de ella, se generó un enjambre de muestras artificiales añadiendo ruido gaussiano independiente a cada uno de los 18 canales, acotando la desviación típica empíricamente para no solapar los clústeres de clases distintas. El dataset resultante (`dataset_completo_40036_v2.csv`) alcanzó aproximadamente **40 036 muestras**, distribuidas equitativamente entre las cinco clases consideradas (Milflores, Retama, Miel de Bosque, Jaramago y Miel Industrial). Este proceso conserva la macroestructura probabilística original, proporcionando el volumen tensorial mínimo requerido para que los algoritmos de optimización por descenso de gradiente puedan iterar sin colapsar en las primeras épocas de entrenamiento.

Enfoque de aprendizaje profundo con destilación de conocimiento (TensorFlow y Keras)

En primera instancia, y con el objetivo de validar la arquitectura base del software, se implementaron redes neuronales densas (*Multi-Layer Perceptron*) haciendo uso de TensorFlow y Keras. El propósito de este bloque algorítmico no se limitó a la búsqueda de precisión absoluta inmediata, sino que sirvió para cumplir con hitos críticos de integración continua:

- **Validación de escalabilidad arquitectónica:** Asegurar que la capa modular de la aplicación de escritorio (GUI, base de datos local

embebida y controladores lógicos) estuviera completamente preparada para instanciar el intérprete de Python, cargar grafos computacionales complejos en memoria y realizar inferencias tensoriales sin bloquear el hilo principal de ejecución gráfica (*Main Thread*).

- **Evaluación de hiperparámetros y optimización:** Analizar la convergencia empírica del optimizador Adam [15] y monitorizar la velocidad de propagación del gradiente utilizando la función de activación Swish, comprobando su superioridad matemática frente a arquitecturas clásicas ReLU en el modelado de espacios de características espectrales continuas.

El modelo final integrado en producción aplica un paradigma avanzado de **destilación de conocimiento** (*knowledge distillation*) en dos etapas:

1. **Generación de *soft labels* con XGBoost.** Se entrenó un clasificador `XGBClassifier` mediante validación cruzada y optimización exhaustiva (`GridSearchCV`) sobre su profundidad máxima de árbol, tasa de aprendizaje (*learning rate*) y ratios de submuestreo. Las probabilidades predichas por este modelo maestro sobre el conjunto de entrenamiento no son clasificaciones categóricas duras (0 o 1), sino vectores de probabilidades suavizadas (*soft labels*). Estas etiquetas capturan la similitud oculta o solapamiento espectral intrínseco entre clases botánicas afines (el llamado *dark knowledge*).
2. **Entrenamiento de la red neuronal por imitación.** Una red neuronal densa (estudiante) —con capas ocultas de 128, 64 y 32 neuronas— se entrenó exclusivamente para reproducir dichas distribuciones de probabilidad multiclase, minimizando la función de entropía cruzada categórica respecto a las *soft labels* originadas por XGBoost. El modelo resultante (`mejor_modelo.keras`) se compila y almacena junto a un mapeador estructural en formato JSON (`clases.json`), garantizando la decodificación determinista de las inferencias en tiempo de ejecución de la aplicación.

Esta topología permite que la red neuronal —estructuralmente más veloz en la fase de inferencia y matemáticamente más predecible dentro del encapsulado de la aplicación de escritorio— herede los sesgos inductivos protectores y la alta capacidad discriminativa del ensamblado XGBoost, logrando un compromiso tecnológico óptimo entre complejidad de cómputo y resiliencia analítica.

Enfoque de aprendizaje automático clásico: Bosques Aleatorios (`scikit-learn`)

Como contramedida directa a la escasez de datos reales y para proveer una alternativa algorítmica de comprobada fiabilidad, la experimentación se complementó con modelos de aprendizaje supervisado clásico basados en ensamblados de árboles de decisión (*Random Forests*), orquestados bajo la biblioteca `scikit-learn` [18]. En el caso específico de este trabajo se empleó su variante `cuML` [20], perteneciente al ecosistema RAPIDS de NVIDIA, que ofrece una interfaz compatible con `scikit-learn` pero con ejecución acelerada mediante GPU.

La integración técnica de este enfoque aporta dos ventajas defensivas fundamentales frente a la vulnerabilidad del dataset original:

- **Resiliencia estructural ante el ruido:** Al aplicar técnicas de agregación de arranque empírico (*bagging*) y selección estocástica de subconjuntos de canales espectrales para cada división interna de nodo, el algoritmo promedia las decisiones de múltiples árboles matemáticamente ortogonales. Esto neutraliza las fluctuaciones aleatorias del sensor analógico y demuestra una excepcional tolerancia a valores atípicos (*outliers*).
- **Validación cruzada estratificada:** Para explotar eficientemente la información empírica sin sesgar los resultados ni incurrir en fugas de información, se implementó una partición rigurosa de *5-fold cross-validation* sobre las 42 muestras originales. Se seleccionó el parámetro $k = 5$ como el equilibrio idóneo matemático que garantiza conjuntos de validación suficientemente poblados para calcular métricas fiables (como *Precision*, *Recall* y *F1-Score*, esenciales ante posibles desbalances de clase), asegurando que el modelo se evalúe sistemáticamente contra espectros inéditos en su ciclo de entrenamiento.

Modelo	Librería	Exactitud (CV)	T. entrenamiento
Red neuronal densa (MLP)	TensorFlow/Keras	87 %	15 min
Bosques Aleatorios	scikit-learn / cuML	89 %	7 min
XGBoost	xgboost	92 %	7 min
Modelo Híbrido	XGBoost + TensorFlow	91,5 %	10 min

Tabla 5.1: Comparativa de los modelos evaluados en HIVES. La exactitud se obtuvo mediante validación cruzada de 5 pliegues. Los tiempos de entrenamiento corresponden a la ejecución paralela asistida por GPU en Google Colab.

5.3. Análisis crítico de las limitaciones estadísticas del modelo

Gracias al riguroso diseño de ingeniería de software expuesto en los epígrafes anteriores, el sistema HIVES unifica con éxito la solidez operativa que proporcionan los algoritmos clásicos con la escalabilidad estructural proyectada por las redes neuronales. La generación del dataset sintético ha cumplido su propósito fundamental: asentar y validar toda la infraestructura de bases de datos, flujos de serialización de tensores y gestión de memoria requerida para el funcionamiento estable del aplicativo en fase de producción.

No obstante, desde la estricta perspectiva del rigor científico, metodológico y metrológico, es una exigencia académica señalar que las métricas de exactitud obtenidas (Tabla 5.1) deben interpretarse bajo un prisma puramente arquitectónico y no como un indicador concluyente de precisión agronómica comercial. El conjunto empírico base, compuesto por **única-mente 42 muestras reales**, representa un volumen estadístico significativamente insuficiente frente a las necesidades de convergencia empírica que dictan los estándares del *Deep Learning* aplicado a la espectroscopía de alta resolución [8].

La veracidad de los porcentajes de acierto algorítmico reportados, alcanzando un 92 % máximo en XGBoost (modelo empleado como base para la primera fase del algoritmo híbrido del desarrollo final) se encuentra sujeta a severas limitaciones estructurales que deben ser reconocidas y documentadas:

- **Riesgo crítico de sobreajuste inducido (Fuga de datos indirecta):** Si bien el ruido gaussiano inyectado en el preprocesamiento preserva la matriz de covarianza de la firma real, las cerca de 40 000 muestras artificiales generadas no materializan nuevas variaciones botánicas reales (como diferencias en el estrés hídrico de la planta, maduración enzimática de la miel o variabilidad estacional), sino simples traslaciones y dispersiones matemáticas locales. Al entrenar y validar sobre transformaciones sintéticas derivadas de solo 42 puntos de origen físico, existe una alta probabilidad teórica de que los modelos de ensamblado estén reconociendo la "huella dactilar analógica" de esas 42 muestras específicas y sus vecindades esféricas inmediatas, inflando de forma indirecta el rendimiento reportado ante métricas de validación rutinaria.
- **Variabilidad botánica no modelada linealmente:** La huella composicional de una clase de miel específica (por ejemplo, alteraciones sutiles en los picos de absorción de polifenoles o flavonoides dependiendo de la microrregión geográfica de pecoreo) es un fenómeno termodinámico y bioquímico altamente no lineal. El aumento de datos masivo mediante difusión gaussiana asume intrínsecamente una variación lineal e isotrópica en la sensibilidad de los canales del sensor, lo que no refleja de manera fiel las desviaciones multidimensionales reales que ocurrirían al recolectar mieles de la misma especie botánica en contextos ambientales dispares.
- **Ausencia de un conjunto de prueba empírico independiente (*Hold-out Test Set*):** La literatura metodológica de validación algorítmica exige de forma ineludible la segregación de un conjunto de prueba completamente ciego y de procedencia empírica certificada. Debido a la restricción logística insalvable impuesta por las 42 muestras disponibles, su división matemática fragmentaría excesivamente la representación de las clases minoritarias (resultando hipotéticamente en apenas 1 o 2 muestras por clase destinadas a la fase de testeo). Esta fragmentación hace virtualmente imposible la extracción de un intervalo de confianza estadísticamente significativo o de una matriz de confusión representativa que soporte escrutinio pericial.

En conclusión, la argumentación de este capítulo corrobora que el proyecto constituye una **prueba de concepto tecnológica exitosa y una validación funcional integral del diseño arquitectónico del software de HIVES**. Las pruebas demuestran de forma incontestable que la cadena

de ingeniería completa —desde la excitación del fotodiodo en el encapsulado hardware de la capa física, pasando por la transcodificación analógico-digital, hasta la ingesta, serialización y el renderizado final de la inferencia multiclase en la interfaz visual de Python— opera sin cuellos de botella asíncronos ni fallos de integración estructurales.

La declaración proactiva de estas limitaciones algorítmicas responde a las buenas prácticas éticas exigibles en la ingeniería de sistemas críticos. Para que el núcleo analítico logre la validez pericial o metrológica requerida para su explotación comercial definitiva en la industria agroalimentaria, el requisito indispensable será la paulatina sustitución de este dataset sintético por una base de datos masiva compuesta por cientos de muestras reales, certificadas unívocamente mediante laboratorio convencional. Esta limitación de alcance asienta las bases directas y pragmáticas para la futura hoja de ruta y continuación investigadora del proyecto, tal y como se detalla en profundidad en el Capítulo 7 de conclusiones y líneas de trabajo futuro.

Trabajos relacionados

El desarrollo de sistemas orientados a la clasificación y control de calidad de productos agroalimentarios mediante técnicas ópticas no destructivas constituye un área de investigación de gran dinamismo en la intersección de la ingeniería ciberfísica, la quimiometría y la inteligencia artificial. En este capítulo se expone una revisión del estado del arte, clasificando los trabajos previos en función de sus aportaciones al análisis espectroscópico de la miel, la detección algorítmica de adulteraciones y el empleo de infraestructuras modulares de adquisición de datos.

6.1. Caracterización física y colorimetría espectral tradicional

El uso de propiedades ópticas para inferir la calidad y el origen de la miel cuenta con precedentes consolidados en la literatura científica. Inicialmente, las investigaciones se centraban en la medición de parámetros físicos discretos correlacionados con la composición bromatológica del fluido.

Un trabajo de referencia en este ámbito es el desarrollado por Simal Lozano et al. [21], quienes profundizaron en la determinación analítica del color y la turbidez como indicadores directos de las propiedades organolépticas y el origen de las mieles. Estas aproximaciones clásicas demostraron que el espectro visible absorbe de forma heterogénea dependiendo de los pigmentos (carotenoides, flavonoides) y los elementos en suspensión (trazas de polen o microburbujas), sentando las bases ópticas que justifican el uso de fotodiodos para registrar firmas espectrales.

No obstante, estos estudios tradicionales dependían de instrumentación de laboratorio compleja, como espectrofotómetros de mesa de alta resolución, y de técnicas de procesamiento puramente estadísticas (regresiones lineales), careciendo de la automatización y capacidad de generalización que proveen los flujos de software actuales.

6.2. Espectroscopía infrarroja y detección de adulteraciones

Con el avance de los sensores optoelectrónicos, el foco de la comunidad científica se orientó hacia el uso de rangos electromagnéticos más amplios, especialmente el infrarrojo cercano (NIR) y medio, debido a su sensibilidad ante las vibraciones moleculares de los enlaces químicos presentes en los azúcares (glucosa, fructosa) y el agua.

En esta línea, la investigación de Guillén Cueto [10] detalla el éxito de la aplicación de la técnica NIRS para la autenticación de mieles adulteradas. Su trabajo demuestra que la radiación en el infrarrojo cercano permite capturar firmas moleculares complejas que reflejan alteraciones sutiles en la matriz de la miel, causadas por la adición fraudulenta de siropes exógenos (como jarabes de maíz o caña de azúcar). El procesamiento de estos espectros NIR de alta dimensionalidad mediante modelos quimiométricos avanzados abrió la puerta al uso de algoritmos de clasificación supervisada para la inspección alimentaria automatizada.

Asimismo, estudios consolidados en revistas especializadas de química alimentaria, como los publicados en *Food Chemistry* [13], corroboran que el análisis multifrecuencial que abarca tanto el espectro ultravioleta como las longitudes de onda del infrarrojo cercano proporciona vectores de características con un poder discriminativo muy superior al de las técnicas ópticas restringidas al ojo humano. Estas investigaciones justifican la arquitectura de almacenamiento y preparación tensorial implementada en HIVES, diseñada para procesar arrays de 18 canales que solapan precisamente estas regiones críticas del espectro.

6.3. Evolución hacia repositorios abiertos y modelos de Machine Learning

En el ámbito estrictamente informático, el auge del *Data Science* ha propiciado la proliferación de conjuntos de datos de libre acceso enfocados

en la seguridad alimentaria. Un ejemplo de este paradigma es el *Adulterated Honey Dataset* alojado en la plataforma de ciencia de datos Kaggle [7], el cual recopila perfiles analíticos y físico-químicos masivos de muestras de miel destinados al entrenamiento de redes neuronales y clasificadores supervisados.

La disponibilidad de estos entornos de pruebas digitales ha permitido a la comunidad de ingenieros de software evaluar la convergencia de algoritmos en entornos de computación distribuidos o en la nube, empleando herramientas como **Google Colab**. Sin embargo, la gran mayoría de estos trabajos teóricos adolecen de una desconexión con el entorno físico: se nutren exclusivamente de datos tabulares preexistentes en repositorios, sin proponer una arquitectura de integración extremo a extremo con hardware de captura en tiempo real.

6.4. Posicionamiento y aportación del sistema HIVES

La revisión del estado del arte evidencia una brecha tecnológica clara entre dos mundos: por un lado, las soluciones laborales de alta precisión basadas en espectroscopía pesada [10, 13], inaccesibles para el pequeño productor apícola debido a sus elevados costes operativos; y por otro lado, los desarrollos de software algorítmico basados en repositorios estáticos [7], desvinculados de la toma de medidas empíricas.

El sistema HIVES aporta valor de ingeniería precisamente como un puente ciberfísico modular. Al integrar un microsensor espectrométrico de bajo coste con una aplicación de escritorio robusta desarrollada bajo principios de modularidad (MVC) y concurrencia (PyQt6), este proyecto demuestra la viabilidad de trasladar los principios de la quimiometría espectral computacional desde los laboratorios tradicionales directamente hacia un dispositivo portátil y económico.

Asimismo, la inclusión de un flujo algorítmico de evaluación dual que dota a la aplicación de un motor basado en Bosques Aleatorios (**scikit-learn**) robusto ante la escasez de muestras físicas, manteniendo simultáneamente los enlaces de carga preparados para redes neuronales densas (**TensorFlow**), representa una solución de compromiso metodológico realista alineada con el ciclo de vida del software en escenarios de explotación industrial reales.

6.5. Técnica Híbrida de modelos de ML (XGBoost + TensorFlow)

El enfoque metodológico adoptado en HIVES no es exclusivo del ámbito de la espectrometría. Kashani et al. propusieron un marco híbrido que combina redes neuronales profundas (*Deep Neural Networks*, DNN) y XGBoost para la detección temprana de diabetes de tipo 2, obteniendo una precisión del 94 % con el modelo híbrido, frente al 91 % del DNN aislado y el 89 % del XGBoost independiente. Aunque el dominio de aplicación difiere radicalmente del presente proyecto, el estudio valida la estrategia de combinar modelos de aprendizaje profundo con métodos de *gradient boosting* cuando se dispone de datos tabulares de naturaleza espectral o clínica. Resulta especialmente relevante que el pipeline completo fue implementado en Python 3.10 con TensorFlow 2.12 y XGBoost 2.0, coincidiendo con el entorno tecnológico empleado en HIVES [14].

El trabajo ilustra asimismo el valor de incorporar técnicas de inteligencia artificial explicable (XAI) mediante SHAP para justificar las predicciones del modelo, una línea de mejora que se identifica como trabajo futuro en el presente proyecto.

Conclusiones y Líneas de trabajo futuras

En este último capítulo se exponen las conclusiones generales derivadas del desarrollo del sistema HIVES, analizando el grado de consecución de los objetivos planteados y evaluando críticamente los retos de ingeniería de software superados durante la integración. Asimismo, se definen las líneas de investigación y desarrollo futuras orientadas a la evolución tecnológica del dispositivo y del software predictivo.

7.1. Conclusiones

Tras la finalización del proyecto, la conclusión principal es que se han cumplido satisfactoriamente la totalidad de los objetivos técnicos y funcionales establecidos en la especificación inicial del sistema. Desde una perspectiva académica y profesional, el desarrollo del analizador HIVES ha constituido un hito formativo de gran valor, al haber permitido consolidar y poner en práctica de manera unificada en un único proyecto tangible los conceptos transversales adquiridos a lo largo del Grado en Ingeniería Informática (tales como la arquitectura de software, la gestión de bases de datos, el desarrollo de interfaces gráficas reactivas y la aplicación práctica de la inteligencia artificial).

En lo relativo al comportamiento operativo y al rendimiento de la solución, el sistema ha demostrado una alta fiabilidad en la adquisición y procesamiento de datos. No obstante, la integración de la capa de comunicación hardware-software supuso uno de los mayores retos del proyecto. En las fases iniciales de pruebas, surgieron problemas de sincronización en la transmisión de flujos serie desde el microcontrolador hacia la estación de

trabajo. El principal obstáculo detectado se centró en escenarios de desconexiones intermitentes debidos a conflictos en el acceso concurrente al puerto serie virtual. Durante el flujo de desarrollo, la monitorización simultánea mediante el monitor serie del *Arduino IDE* y la escucha activa del controlador implementado en *Visual Studio Code* provocaban la saturación y el bloqueo del buffer serie del hardware. Este inconveniente técnico se solventó con éxito mediante la depuración estricta del protocolo de comunicación y la correcta delegación de tareas asíncronas a través de hilos de ejecución independientes, garantizando la resiliencia de la interfaz gráfica y la estabilidad del canal serie en la versión final de producción.

Finalmente, cabe destacar el impacto formativo derivado de las limitaciones operativas del conjunto de datos. La necesidad de explorar y verificar el comportamiento de redes neuronales artificiales densas con un volumen de 42 muestras impulsó la investigación activa en el ámbito de la computación y el entrenamiento en la nube. La utilización de entornos cloud avanzados como *Google Colab* permitió experimentar con el rendimiento algorítmico de forma ágil y deslocalizada. Este campo técnico, que habitualmente cuenta con una exposición limitada dentro de los planes de estudio de la titulación, ha resultado ser un área de especialización sumamente interesante, aportando conocimientos críticos sobre las infraestructuras para proyectos complejos de computación intensiva.

7.2. Líneas de trabajo futuras

A partir de los resultados y de la experiencia práctica obtenida en este Trabajo de Fin de Grado, se proponen las siguientes líneas de trabajo futuras para expandir las capacidades funcionales y tecnológicas del sistema:

- **Ampliación del conjunto de datos empíricos y revalidación del clasificador:** El objetivo prioritario e inmediato consiste en expandir de forma sustancial el volumen del *dataset* de firmas espectrales reales obtenidas directamente con el sensor AS7265x. Tal y como se analiza en detalle en la Sección 5.2 de la memoria, las métricas de clasificación reportadas en el presente trabajo fueron obtenidas sobre un conjunto de datos mayoritariamente sintético —generado mediante difusión gaussiana a partir de únicamente 42 muestras reales—, por lo que su validez en condiciones reales es limitada. La incorporación continua de nuevas muestras certificadas mediante análisis melisopalinológico independiente permitirá, en primer lugar, reentrenar el clasificador

sobre datos suficientes y, en segundo lugar, disponer de un conjunto de prueba independiente con el que evaluar el rendimiento real del sistema. Solo a partir de ese punto podrá considerarse que las métricas de exactitud del clasificador constituyen evidencia válida para un contexto de aplicación práctica o comercial.

- **Evolución del hardware espectrométrico (Bandas extremas UV e IR):** El hardware actual (AS7265x) ha demostrado una excelente capacidad para extraer características y predecir parámetros integrados en el modelo de *Machine Learning* dentro del rango de 410 nm a 940 nm. No obstante, estudios académicos consolidados [13] demuestran que el análisis de muestras de miel en longitudes de onda situadas por debajo del espectro visible (ultravioleta profundo) y muy por encima de este (infrarrojo medio y cercano, NIRS [10]) revela información altamente discriminativa sobre compuestos fenólicos y origen floral. Por ello, una evolución natural del prototipo sería la integración de sensores complementarios que abarquen estas bandas extremas.
- **Migración hacia un ecosistema móvil y portátil:** Con el fin de maximizar la usabilidad en entornos de campo (como en explotaciones apícolas productivas), se contempla la sustitución de la arquitectura de escritorio local por un despliegue portátil. Esta evolución requerirá el desarrollo de una aplicación móvil multiplataforma y la reingeniería del módulo de comunicaciones para reemplazar la conexión física USB por un protocolo inalámbrico mediante tecnología Bluetooth, dotando al hardware de una autonomía completa.

Bibliografía

- [1] ams AG. *AS7265x Triad Spectroscopy Sensor: Datasheet*. ams AG, Premstätten, Austria, 2018. Disponible en: https://cdn.sparkfun.com/assets/c/2/9/0/a/AS7265x_Datasheet.pdf.
- [2] Jan Axelson. *Serial Port Complete: The Developer's Guide to USB and Serial Communications*. Lakeview Research, Madison, 2 edition, 2007.
- [3] Esben Jannik Bjerrum. Data augmentation of spectral data for convolutional neural network (CNN) based deep chemometrics. *arXiv preprint*, 2017.
- [4] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- [5] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794. ACM, 2016.
- [6] Exiplast. Iluminación natural en el diseño luminotécnico, 2026. [Accedido: 3-junio-2026].
- [7] Peyush Gedela. Adulterated honey dataset. <https://www.kaggle.com/datasets/peyushgedela/adulterated-honey-dataset>, 2024. Consultado: junio de 2026.
- [8] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, Cambridge, 2016. Disponible en: <https://www.deeplearningbook.org>.

- [9] Google Inc. TensorFlow: Large-scale machine learning on heterogeneous distributed systems. Documentación oficial, 2024. Disponible en: https://www.tensorflow.org/api_docs.
- [10] Lucía Guillén Cueto. Aplicación de la técnica nirs para la autenticación de mieles adulteradas. Trabajo de fin de grado, Universidad de Cádiz, 2017. Disponible en: <https://rodin.uca.es/bitstream/handle/10498/20573/>.
- [11] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, Nueva York, 2 edition, 2009. Disponible en: <https://hastie.su.domains/ElemStatLearn/>.
- [12] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network, 2015.
- [13] Romdhane Karoui. The use of front face fluorescence spectroscopy to classify the botanical origin of honey samples produced in switzerland. *Food Chemistry*, 2007. Disponible en: <https://www.sciencedirect.com/science/article/abs/pii/S0308814606000951>.
- [14] Mostafa Kashani, Sedigheh Barzehkar, and Mehran Kamani. A hybrid deep learning and explainable AI framework for early detection of type 2 diabetes: A multi-factor approach. *Shiraz E-Medical Journal*, 27(2):e166066, 2025. Consultado: junio 2025.
- [15] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint*, arXiv:1412.6980, 2015. Disponible en: <https://arxiv.org/abs/1412.6980>.
- [16] Jean Louveaux, Anna Maurizio, and Günter Vorwohl. Methods of melissopalynology. *Bee World*, 59(4):139–157, 1978.
- [17] Parlamento Europeo and Consejo de la Unión Europea. Directiva 2001/110/ce del consejo relativa a la miel. Diario Oficial de las Comunidades Europeas, L 10, 2001. Disponible en: <https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=CELEX%3A32001L0110>.
- [18] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. Disponible en: <https://jmlr.org/papers/v12/pedregosa11a.html>.

- [19] Prajit Ramachandran, Barret Zoph, and Quoc V. Le. Searching for activation functions. *arXiv preprint*, arXiv:1710.05941, 2017. Disponible en: <https://arxiv.org/abs/1710.05941>.
- [20] RAPIDS Development Team. cuML: GPU-accelerated machine learning library. <https://github.com/rapidsai/cuml>, 2023.
- [21] Jesús Simal Lozano et al. Determinación del color y de la turbidez en las mieles. *Anales de Bromatología*, 1991. Disponible en: <https://www.researchgate.net/publication/314057983>.
- [22] Douglas A. Skoog, F. James Holler, and Stanley R. Crouch. *Principios de análisis instrumental*. Cengage Learning, Ciudad de México, 7 edition, 2018.
- [23] SparkFun Electronics. Sparkfun triad spectroscopy sensor - as7265x (qwiic), 2026. [Accedido: 3-junio-2026].